# Specification-Driven Development (SDD)

## The Power Inversion

For decades, code has been king. Specifications served code-they were the scaffolding we built and then discarded once the "real work" of coding began. We wrote PRDs to guide development, created design docs to inform implementation, drew diagrams to visualize architecture. But these were always subordinate to the code itself. Code was truth. Everything else was, at best, good intentions. Code was the source of truth, and as it moved forward, specs rarely kept pace. As the asset (code) and the implementation are one, it's not easy to have a parallel implementation without trying to build from the code.

Spec-Driven Development (SDD) inverts this power structure. Specifications don't serve code-code serves specifications. The Product Requirements Document (PRD) isn't a guide for implementation; it's the source that generates implementation. Technical plans aren't documents that inform coding; they're precise definitions that produce code. This isn't an incremental improvement to how we build software. It's a fundamental rethinking of what drives development.

The gap between specification and implementation has plagued software development since its inception. We've tried to bridge it with better documentation, more detailed requirements, stricter processes. These approaches fail because they accept the gap as inevitable. They try to narrow it but never eliminate it. SDD eliminates the gap by making specifications and their concrete implementation plans born from the specification executable. When specifications and implementation plans generate code, there is no gap-only transformation.

This transformation is now possible because AI can understand and implement complex specifications, and create detailed implementation plans. But raw AI generation without structure produces chaos. SDD provides that structure through specifications and subsequent implementation plans that are precise, complete, and unambiguous enough to generate working systems. The specification becomes the primary artifact. Code becomes its expression (as an implementation from the implementation plan) in a particular language and framework.

In this new world, maintaining software means evolving specifications. The intent of the development team is expressed in natural language ("**intent-driven development**"), design assets, core principles and other guidelines. The **lingua franca** of development moves to a higher level, and code is the last-mile approach.

Debugging means fixing specifications and their implementation plans that generate incorrect code. Refactoring means restructuring for clarity. The entire development workflow reorganizes around specifications as the central source of truth, with implementation plans and code as the continuously regenerated output. Updating apps with new features or creating a new parallel implementation because we are creative beings, means revisiting the specification and creating new implementation plans. This process is therefore a 0 -> 1, (1', ..), 2, 3, N.

The development team focuses in on their creativity, experimentation, their critical thinking.

## The SDD Workflow in Practice

The workflow begins with an idea-often vague and incomplete. Through iterative dialogue with AI, this idea becomes a comprehensive PRD. The AI asks clarifying questions, identifies edge cases, and helps define precise acceptance criteria. What might take days of meetings and documentation in traditional development happens in hours of focused specification work. This transforms the traditional SDLC-requirements and design become continuous activities rather than discrete phases. This is supportive of a **team process**, where team-reviewed specifications are expressed and versioned, created in branches, and merged.

When a product manager updates acceptance criteria, implementation plans automatically flag affected technical decisions. When an architect discovers a better pattern, the PRD updates to reflect new possibilities.

Throughout this specification process, research agents gather critical context. They investigate library compatibility, performance benchmarks, and security implications. Organizational constraints are discovered and applied automatically-your company's database standards, authentication requirements, and deployment policies seamlessly integrate into every specification.

From the PRD, AI generates implementation plans that map requirements to technical decisions. Every technology choice has documented rationale. Every architectural decision traces back to specific requirements. Throughout this process, consistency validation continuously improves quality. AI analyzes specifications for ambiguity, contradictions, and gaps-not as a one-time gate, but as an ongoing refinement.

Code generation begins as soon as specifications and their implementation plans are stable enough, but they do not have to be "complete." Early generations might be exploratory-testing whether the specification makes sense in practice. Domain concepts become data models. User stories become API endpoints. Acceptance scenarios become tests. This merges development and testing through specification-test scenarios aren't written after code, they're part of the specification that generates both implementation and tests.

The feedback loop extends beyond initial development. Production metrics and incidents don't just trigger hotfixes-they update specifications for the next regeneration. Performance bottlenecks become new non-functional requirements. Security vulnerabilities become constraints that affect all future generations. This iterative dance between specification, implementation, and operational reality is where true understanding emerges and where the traditional SDLC transforms into a continuous evolution.

## Why SDD Matters Now

Three trends make SDD not just possible but necessary:

First, AI capabilities have reached a threshold where natural language specifications can reliably generate working code. This isn't about replacing developers-it's about amplifying their effectiveness by automating the mechanical translation from specification to implementation. It can amplify exploration and creativity, support "start-over" easily, and support addition, subtraction, and critical thinking.

Second, software complexity continues to grow exponentially. Modern systems integrate dozens of services, frameworks, and dependencies. Keeping all these pieces aligned with original intent through manual processes becomes increasingly difficult. SDD provides systematic alignment through specification-driven generation. Frameworks may evolve to provide AI-first support, not human-first support, or architect around reusable components.

Third, the pace of change accelerates. Requirements change far more rapidly today than ever before. Pivoting is no longer exceptional-it's expected. Modern product development demands rapid iteration based on user feedback, market conditions, and competitive pressures. Traditional development treats these changes as

disruptions. Each pivot requires manually propagating changes through documentation, design, and code. The result is either slow, careful updates that limit velocity, or fast, reckless changes that accumulate technical debt.

SDD can support what-if/simulation experiments: "If we need to re-implement or change the application to promote a business need to sell more T-shirts, how would we implement and experiment for that?"

SDD transforms requirement changes from obstacles into normal workflow. When specifications drive implementation, pivots become systematic regenerations rather than manual rewrites. Change a core requirement in the PRD, and affected implementation plans update automatically. Modify a user story, and corresponding API endpoints regenerate. This isn't just about initial development-it's about maintaining engineering velocity through inevitable changes.

## Core Principles

**Specifications as the Lingua Franca**: The specification becomes the primary artifact. Code becomes its expression in a particular language and framework. Maintaining software means evolving specifications.

**Executable Specifications**: Specifications must be precise, complete, and unambiguous enough to generate working systems. This eliminates the gap between intent and implementation.

**Continuous Refinement**: Consistency validation happens continuously, not as a one-time gate. AI analyzes specifications for ambiguity, contradictions, and gaps as an ongoing process.

**Research-Driven Context**: Research agents gather critical context throughout the specification process, investigating technical options, performance implications, and organizational constraints.

**Bidirectional Feedback**: Production reality informs specification evolution. Metrics, incidents, and operational learnings become inputs for specification refinement.

**Branching for Exploration**: Generate multiple implementation approaches from the same specification to explore different optimization targets-performance, maintainability, user experience, cost.

## Implementation Approaches

Today, practicing SDD requires assembling existing tools and maintaining discipline throughout the process. The methodology can be practiced with:

- AI assistants for iterative specification development
- Research agents for gathering technical context
- Code generation tools for translating specifications to implementation
- Version control systems adapted for specification-first workflows
- Consistency checking through AI analysis of specification documents

The key is treating specifications as the source of truth, with code as the generated output that serves the specification rather than the other way around.

## Streamlining SDD with Commands

The SDD methodology is significantly enhanced through three powerful commands that automate the specification -> planning -> tasking workflow:

### The `/specify.specify` Command

This command transforms a simple feature description (the user-prompt) into a complete, structured specification with automatic repository management:

1. **Automatic Feature Numbering**: Scans existing specs to determine the next feature number (e.g., 001, 002, 003)
2. **Branch Creation**: Generates a semantic branch name from your description and creates it automatically
3. **Template-Based Generation**: Copies and customizes the feature specification template with your requirements
4. **Directory Structure**: Creates the proper `specs/[branch-name]/` structure for all related documents

### The `/specify.plan` Command

Once a feature specification exists, this command creates a comprehensive implementation plan:

1. **Specification Analysis**: Reads and understands the feature requirements, user stories, and acceptance criteria
2. **Constitutional Compliance**: Ensures alignment with project constitution and architectural principles
3. **Technical Translation**: Converts business requirements into technical architecture and implementation details
4. **Detailed Documentation**: Generates supporting documents for data models, API contracts, and test scenarios
5. **Quickstart Validation**: Produces a quickstart guide capturing key validation scenarios

### The `/specify.tasks` Command

After a plan is created, this command analyzes the plan and related design documents to generate an executable task list:

1. **Inputs**: Reads `plan.md` (required) and, if present, `data-model.md`, `contracts/`, and `research.md`
2. **Task Derivation**: Converts contracts, entities, and scenarios into specific tasks
3. **Parallelization**: Marks independent tasks `[P]` and outlines safe parallel groups
4. **Output**: Writes `tasks.md` in the feature directory, ready for execution by a Task agent

### Example: Building a Chat Feature

Here's how these commands transform the traditional development workflow:

**Traditional Approach:**

```
1. Write a PRD in a document (2-3 hours)
2. Create design documents (2-3 hours)
3. Set up project structure manually (30 minutes)
4. Write technical specifications (3-4 hours)
5. Create test plans (2 hours)
Total: ~12 hours of documentation work
```

**SDD with Commands Approach:**

```
# Step 1: Create the feature specification (5 minutes)
/specify.specify Real-time chat system with message history and user presence

# This automatically:
# - Creates branch "003-chat-system"
# - Generates specs/003-chat-system/spec.md
# - Populates it with structured requirements

# Step 2: Generate implementation plan (5 minutes)
/specify.plan WebSocket for real-time messaging, PostgreSQL for history, Redis for
presence

# Step 3: Generate executable tasks (5 minutes)
/specify.tasks

# This automatically creates:
# - specs/003-chat-system/plan.md
# - specs/003-chat-system/research.md (WebSocket library comparisons)
# - specs/003-chat-system/data-model.md (Message and User schemas)
# - specs/003-chat-system/contracts/ (WebSocket events, REST endpoints)
# - specs/003-chat-system/quickstart.md (Key validation scenarios)
# - specs/003-chat-system/tasks.md (Task list derived from the plan)
```

In 15 minutes, you have:

- A complete feature specification with user stories and acceptance criteria
- A detailed implementation plan with technology choices and rationale
- API contracts and data models ready for code generation
- Comprehensive test scenarios for both automated and manual testing
- All documents properly versioned in a feature branch

### The Power of Structured Automation

These commands don't just save time-they enforce consistency and completeness:

1. **No Forgotten Details**: Templates ensure every aspect is considered, from non-functional requirements to error handling
2. **Traceable Decisions**: Every technical choice links back to specific requirements
3. **Living Documentation**: Specifications stay in sync with code because they generate it
4. **Rapid Iteration**: Change requirements and regenerate plans in minutes, not days

The commands embody SDD principles by treating specifications as executable artifacts rather than static documents. They transform the specification process from a necessary evil into the driving force of development.

### Template-Driven Quality: How Structure Constrains LLMs for Better Outcomes

The true power of these commands lies not just in automation, but in how the templates guide LLM behavior toward higher-quality specifications. The templates act as sophisticated prompts that constrain the LLM's output in productive ways:

**1. Preventing Premature Implementation Details**

The feature specification template explicitly instructs:

```
- [OK] Focus on WHAT users need and WHY
- [X] Avoid HOW to implement (no tech stack, APIs, code structure)
```

This constraint forces the LLM to maintain proper abstraction levels. When an LLM might naturally jump to "implement using React with Redux," the template keeps it focused on "users need real-time updates of their data." This separation ensures specifications remain stable even as implementation technologies change.

**2. Forcing Explicit Uncertainty Markers**

Both templates mandate the use of `[NEEDS CLARIFICATION]` markers:

```
When creating this spec from a user prompt:
1. **Mark all ambiguities**: Use [NEEDS CLARIFICATION: specific question]
2. **Don't guess**: If the prompt doesn't specify something, mark it
```

This prevents the common LLM behavior of making plausible but potentially incorrect assumptions. Instead of guessing that a "login system" uses email/password authentication, the LLM must mark it as `[NEEDS CLARIFICATION: auth method not specified - email/password, SSO, OAuth?]`.

**3. Structured Thinking Through Checklists**

The templates include comprehensive checklists that act as "unit tests" for the specification:

```
### Requirement Completeness
- [ ] No [NEEDS CLARIFICATION] markers remain
- [ ] Requirements are testable and unambiguous
- [ ] Success criteria are measurable
```

These checklists force the LLM to self-review its output systematically, catching gaps that might otherwise slip through. It's like giving the LLM a quality assurance framework.

**4. Constitutional Compliance Through Gates**

The implementation plan template enforces architectural principles through phase gates:

```
### Phase -1: Pre-Implementation Gates
#### Simplicity Gate (Article VII)
- [ ] Using <=3 projects?
- [ ] No future-proofing?
#### Anti-Abstraction Gate (Article VIII)
```

```
- [ ] Using framework directly?
- [ ] Single model representation?
```

These gates prevent over-engineering by making the LLM explicitly justify any complexity. If a gate fails, the LLM must document why in the "Complexity Tracking" section, creating accountability for architectural decisions.

**5. Hierarchical Detail Management**

The templates enforce proper information architecture:

```
**IMPORTANT**: This implementation plan should remain high-level and readable.
Any code samples, detailed algorithms, or extensive technical specifications
must be placed in the appropriate `implementation-details/` file
```

This prevents the common problem of specifications becoming unreadable code dumps. The LLM learns to maintain appropriate detail levels, extracting complexity to separate files while keeping the main document navigable.

**6. Test-First Thinking**

The implementation template enforces test-first development:

```
### File Creation Order
1. Create `contracts/` with API specifications
2. Create test files in order: contract -> integration -> e2e -> unit
3. Create source files to make tests pass
```

This ordering constraint ensures the LLM thinks about testability and contracts before implementation, leading to more robust and verifiable specifications.

**7. Preventing Speculative Features**

Templates explicitly discourage speculation:

```
- [ ] No speculative or "might need" features
- [ ] All phases have clear prerequisites and deliverables
```

This stops the LLM from adding "nice to have" features that complicate implementation. Every feature must trace back to a concrete user story with clear acceptance criteria.

## The Compound Effect

These constraints work together to produce specifications that are:

- **Complete**: Checklists ensure nothing is forgotten
- **Unambiguous**: Forced clarification markers highlight uncertainties
- **Testable**: Test-first thinking baked into the process
- **Maintainable**: Proper abstraction levels and information hierarchy
- **Implementable**: Clear phases with concrete deliverables

The templates transform the LLM from a creative writer into a disciplined specification engineer, channeling its capabilities toward producing consistently high-quality, executable specifications that truly drive development.

## The Constitutional Foundation: Enforcing Architectural Discipline

At the heart of SDD lies a constitution-a set of immutable principles that govern how specifications become code. The constitution (`memory/constitution.md`) acts as the architectural DNA of the system, ensuring that every generated implementation maintains consistency, simplicity, and quality.

### The Nine Articles of Development

The constitution defines nine articles that shape every aspect of the development process:

#### Article I: Library-First Principle

Every feature must begin as a standalone library-no exceptions. This forces modular design from the start:

```
Every feature in Specify MUST begin its existence as a standalone library.
No feature shall be implemented directly within application code without
first being abstracted into a reusable library component.
```

This principle ensures that specifications generate modular, reusable code rather than monolithic applications. When the LLM generates an implementation plan, it must structure features as libraries with clear boundaries and minimal dependencies.

#### Article II: CLI Interface Mandate

Every library must expose its functionality through a command-line interface:

```
All CLI interfaces MUST:
- Accept text as input (via stdin, arguments, or files)
- Produce text as output (via stdout)
- Support JSON format for structured data exchange
```

This enforces observability and testability. The LLM cannot hide functionality inside opaque classes-everything must be accessible and verifiable through text-based interfaces.

#### Article III: Test-First Imperative

The most transformative article-no code before tests:

```
This is NON-NEGOTIABLE: All implementation MUST follow strict Test-Driven
Development.
No implementation code shall be written before:
1. Unit tests are written
2. Tests are validated and approved by the user
3. Tests are confirmed to FAIL (Red phase)
```

This completely inverts traditional AI code generation. Instead of generating code and hoping it works, the LLM must first generate comprehensive tests that define behavior, get them approved, and only then generate implementation.

**Articles VII & VIII: Simplicity and Anti-Abstraction**

These paired articles combat over-engineering:

```
Section 7.3: Minimal Project Structure
- Maximum 3 projects for initial implementation
- Additional projects require documented justification

Section 8.1: Framework Trust
- Use framework features directly rather than wrapping them
```

When an LLM might naturally create elaborate abstractions, these articles force it to justify every layer of complexity. The implementation plan template's "Phase -1 Gates" directly enforce these principles.

**Article IX: Integration-First Testing**

Prioritizes real-world testing over isolated unit tests:

```
Tests MUST use realistic environments:
- Prefer real databases over mocks
- Use actual service instances over stubs
- Contract tests mandatory before implementation
```

This ensures generated code works in practice, not just in theory.

## Constitutional Enforcement Through Templates

The implementation plan template operationalizes these articles through concrete checkpoints:

```
### Phase -1: Pre-Implementation Gates
#### Simplicity Gate (Article VII)
- [ ] Using <=3 projects?
- [ ] No future-proofing?
```

```
#### Anti-Abstraction Gate (Article VIII)
- [ ] Using framework directly?
- [ ] Single model representation?

#### Integration-First Gate (Article IX)
- [ ] Contracts defined?
- [ ] Contract tests written?
```

These gates act as compile-time checks for architectural principles. The LLM cannot proceed without either passing the gates or documenting justified exceptions in the "Complexity Tracking" section.

### The Power of Immutable Principles

The constitution's power lies in its immutability. While implementation details can evolve, the core principles remain constant. This provides:

1. **Consistency Across Time**: Code generated today follows the same principles as code generated next year
2. **Consistency Across LLMs**: Different AI models produce architecturally compatible code
3. **Architectural Integrity**: Every feature reinforces rather than undermines the system design
4. **Quality Guarantees**: Test-first, library-first, and simplicity principles ensure maintainable code

### Constitutional Evolution

While principles are immutable, their application can evolve:

```
Section 4.2: Amendment Process
Modifications to this constitution require:
- Explicit documentation of the rationale for change
- Review and approval by project maintainers
- Backwards compatibility assessment
```

This allows the methodology to learn and improve while maintaining stability. The constitution shows its own evolution with dated amendments, demonstrating how principles can be refined based on real-world experience.

### Beyond Rules: A Development Philosophy

The constitution isn't just a rulebook-it's a philosophy that shapes how LLMs think about code generation:

- **Observability Over Opacity**: Everything must be inspectable through CLI interfaces
- **Simplicity Over Cleverness**: Start simple, add complexity only when proven necessary
- **Integration Over Isolation**: Test in real environments, not artificial ones
- **Modularity Over Monoliths**: Every feature is a library with clear boundaries

By embedding these principles into the specification and planning process, SDD ensures that generated code isn't just functional-it's maintainable, testable, and architecturally sound. The constitution transforms AI from a code generator into an architectural partner that respects and reinforces system design principles.

## The Transformation

This isn't about replacing developers or automating creativity. It's about amplifying human capability by automating mechanical translation. It's about creating a tight feedback loop where specifications, research, and code evolve together, each iteration bringing deeper understanding and better alignment between intent and implementation.

Software development needs better tools for maintaining alignment between intent and implementation. SDD provides the methodology for achieving this alignment through executable specifications that generate code rather than merely guiding it.

---

## SDD and the Specify Toolkit

To operationalize SDD in this repository we rely on the Specify CLI workflow. It layers concrete guardrails (architecture-as-data, versioning rules, Mermaid validation, exhaustive testing) on top of the principles described above.

1. **Foundations (`/specify.constitution`)** captures the non-negotiable principles (quality, security, delivery) in `.specify/memory/constitution.md`. Every downstream command imports it.
2. **Architecture as data (`/specify.architecture.create`)**
    - Generates `specs/architecture/architecture.json` (authoritative model, patch versions within the `1.0.x` range).
    - Writes C1-C7 views, validated Mermaid diagrams, and cross-links (containers <-> components <-> code structure).
    - Builds `c7_tests/tests_overview.md`, an exhaustive suite catalogue (purpose, owners, commands, coverage focus, risks, last run).
    - Use `mmdc` (`npm install -g @mermaid-js/mermaid-cli`) or `./specs/architecture/export_to_pdf.sh` to validate diagrams and produce a PDF.
3. **Specification & planning (`/specify.specify`, `/specify.plan`, `/specify.tasks`)**
    - Align requirements with the architecture model. After significant changes run `/specify.architecture.update` to keep `architecture.json` and the Markdown views in sync.
    - Tasks inherit the constitution and plan context; security and database implications must be captured in the component and code artefacts before work starts.
4. **Implementation & verification (`/specify.implement`, optional `/specify.clarify`, `/specify.analyze`)**
    - Execute the task list while refreshing breadcrumbs, Mermaid diagrams, the tests table, and security notes.
    - `/specify.architecture.update` bumps model/view patch versions (`1.0.x`) and records changes in `architecture_logs.md`.
5. **Operational tooling**
    - `python -m compileall src/specify_cli` quick regression check for the CLI.

- `npm install -g md-to-pdf @mermaid-js/mermaid-cli` plus `./specs/architecture/export_to_pdf.sh` produce deliverable PDFs.
- `mmdc -i <diagram> -o /tmp/out.svg` (or https://mermaid.live) validate Mermaid diagrams before sharing.

Following this workflow keeps specifications, architecture, plans, tasks, code, and tests in permanent alignment. The AI assistant can automate the mechanical work, but guardrails (versioning rules, Mermaid validation, exhaustive test catalogue, security notes) ensure the delivered system remains faithful to the intent captured in your SDD artefacts.